

Table of Contents

Project 3: Packet Quest Arena	1
Project Summary	1
Learning Objectives	1
Problem Description	2
Game Concept	3
Technical Requirements	3
Controlled Use of AI	5
Code Quality Requirements	6
Security Requirements	6
Scalability Requirements	6
Testing Requirements	7
Functional Acceptance Criteria	7
Stretch Goals	9
Expected Deliverables	9
Final Demonstration	10
Definition of Done	10

Project 3: Packet Quest Arena

Project Summary

Packet Quest Arena is a browser-based multiplayer game where players guide packets through a simulated 5G network while avoiding congestion, latency spikes, misconfigured routes, and failing network functions.

The game is part strategy game, part network routing challenge. Players compete or cooperate to keep traffic flowing across a virtual network while incidents and changing load patterns create disruption.

The project reinforces backend design, real-time browser updates, simulation, testing, deployment, scalability, security, and controlled AI-assisted development.

Learning Objectives

By completing this project, graduates should be able to:

1. Build a browser-based multiplayer routing game.

2. Model graph-based network topology and traffic flows.
3. Implement Java Spring Boot APIs for game state, player actions, and scoring.
4. Use design patterns for routing strategies, action handling, and game rules.
5. Use Python to simulate traffic, congestion, or topology changes.
6. Deploy services using Docker and Kubernetes.
7. Implement automated testing across backend, frontend, Python, and system flows.
8. Use SonarQube to improve code quality and security.
9. Design secure server-side controls for multiplayer game actions.
10. Explain scalability limits and mitigation strategies.

Problem Description

Modern 5G networks carry traffic across many network functions and transport paths. Routing decisions, capacity issues, configuration errors, and failures can affect latency, throughput, and customer experience.

In this game, players manage packet flows through a simplified network topology.

The game board contains:

- Network nodes
- Links between nodes
- Packet flows
- Capacity limits
- Latency values
- Failed or degraded components
- Priority traffic types

Players must route traffic from source nodes to destination nodes while maintaining performance.

Different packet types may have different needs:

- Emergency traffic requires low latency.
- Video traffic requires high bandwidth.
- IoT traffic can tolerate delay but not packet loss.
- Control traffic must be reliable.
- Background traffic can be deprioritised.

Players win by delivering traffic efficiently while minimising dropped packets, latency, and congestion.

Game Concept

Game Mode

The game may be implemented as co-operative mode, competitive mode, or team mode. The MVP should support at least two players in the same game session.

Example Gameplay Loop

1. A player creates a packet arena session.
2. Other players join from the browser.
3. The backend generates a network topology.
4. Packet flows begin appearing.
5. Players select routing actions or capacity actions.
6. The simulator updates link load, latency, and packet delivery.
7. Incidents occur, such as link failures or congestion.
8. Scores update based on delivery success, latency, packet loss, and cost.
9. The game ends after a time limit or traffic target.

Example Challenge

Challenge: Emergency traffic must reach Core-1 within 5 seconds.

Available evidence:

- The shortest path is congested.
- An alternate path has more hops but lower current load.
- One link has intermittent packet loss.
- A previous player action increased capacity on a nearby route.

Possible actions:

- Route traffic through the shortest path.
- Route traffic through the lower-load path.
- Drop lower-priority traffic.
- Increase capacity at a cost.
- Wait for congestion to clear.

Expected best action:

Use the lower-load path and optionally deprioritise background traffic if capacity is still insufficient.

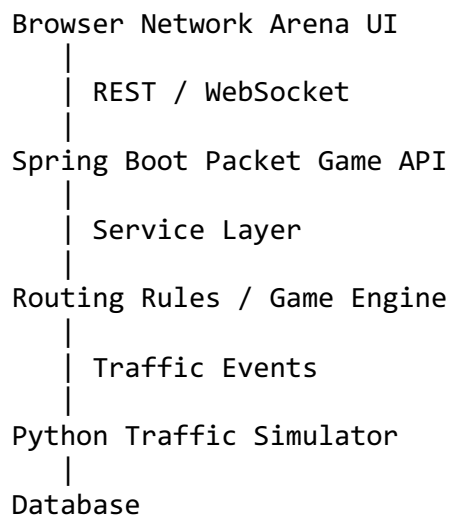
Technical Requirements

Application Architecture

The solution should include:

- Browser-based game interface.
- Java Spring Boot backend.
- Real-time update mechanism.
- Graph-based game model.
- Python traffic simulator.
- Database for sessions, topology, actions, traffic events, and scores.
- Docker support.
- Kubernetes manifests.
- SonarQube integration.

Suggested architecture:



Required Technologies

Backend

The backend must use Java, Spring Boot, REST APIs, WebSocket or equivalent live update approach, persistence, validation, structured logging, and automated tests.

Useful design patterns may include:

- Strategy pattern for routing policies.
- Command pattern for player actions.
- Factory pattern for topology generation.
- Observer or event-driven pattern for traffic updates.

Frontend

The frontend must provide:

- Browser-based game board
- Network topology view

- Link status indicators
- Packet flow display
- Player action controls
- Scoreboard
- Game summary screen

The network topology can be visualised using simple shapes, canvas, SVG, or a frontend graph library.

Python

Python must be used for at least one meaningful component, such as traffic generation, congestion simulation, topology generation, load testing data generation, or post-game route analysis.

Docker

Docker support must include backend, frontend if separate, Python simulator if separate, database, and local development setup.

Kubernetes

Kubernetes manifests must include deployment definitions, services, ConfigMaps or Secrets, readiness and liveness probes, resource requests and limits, and horizontal scaling for at least one service.

Controlled Use of AI

AI may be used to accelerate learning and improve quality, but every output must be reviewed.

Permitted uses include:

- Suggesting route scoring models.
- Drafting test cases.
- Explaining graph algorithms.
- Reviewing code structure.
- Producing documentation drafts.
- Helping identify edge cases.

Not permitted:

- Blindly accepting routing logic.
- Committing code nobody understands.
- Using AI-generated security checks without review.
- Using generated tests without validating expected outcomes.
- Allowing AI to define core rules without team review.

An **AI Usage Log** must be maintained.

Code Quality Requirements

The project must use SonarQube or SonarCloud.

The quality gate should include:

- No blocker or critical issues.
- Meaningful unit test coverage.
- Security hotspots reviewed.
- No hardcoded secrets.
- Low duplication.
- Maintainable code structure.
- Clear naming.
- No unused major logic.

The team should document SonarQube results and how findings were handled.

Security Requirements

Minimum expectations:

- Player actions must be validated server-side.
- Players cannot directly manipulate packet delivery results.
- Scores are calculated by backend rules only.
- APIs validate input.
- No secrets in source control.
- Game session access is controlled.
- Error messages do not expose internal details.
- Invalid routing actions are rejected safely.

Authentication may be lightweight, but authorization rules must be clear.

Scalability Requirements

The solution should support:

- Multiple players.
- Multiple game sessions.
- Increasing number of nodes and links.
- Frequent traffic updates.
- Backend scaling for API and update traffic.
- Reasonable database access patterns.

The team should explain behaviour at:

- 100 players.
- 500 network nodes.
- 2,000 links.
- 100 packet flows per minute.
- 10 simultaneous game sessions.

Testing Requirements

Backend Testing

- Unit tests for routing rules.
- Unit tests for scoring.
- Unit tests for congestion handling.
- API/controller tests.
- Persistence integration tests.
- Negative tests for invalid actions.

Frontend Testing

- Tests for rendering the network view.
- Tests for action submission.
- Tests for score display.
- Tests for link and node status indicators.
- Basic end-to-end gameplay test.

Python Testing

- Tests for traffic generation.
- Tests for topology generation, if used.
- Tests for repeatable simulation behaviour.

System Testing

At least one automated test should prove that:

1. A session can be created.
2. A player can join.
3. Traffic is generated.
4. A player submits a routing action.
5. Packet delivery or score changes correctly.

Functional Acceptance Criteria

Game Session

- A user can create a packet arena session.
- At least two players can join the same session.
- The game has a start and end condition.

- Players can see the current network state.

Network Topology

- The game includes multiple nodes and links.
- Links have capacity, latency, and status.
- Packet flows are generated during gameplay.
- Network conditions change over time.

Player Actions

- Players can choose routing or traffic management actions.
- Actions affect packet delivery, congestion, score, or cost.
- Invalid actions are rejected.
- Different traffic types require different decisions.

Scoring

- Score is calculated by backend logic.
- Score includes at least three factors, such as packet delivery rate, latency, packet loss, cost of actions, and priority traffic protection.
- Score is visible in the browser.

Browser Interface

- The UI shows the topology.
- The UI shows active packet flows.
- The UI shows link or node health.
- The UI allows action submission.
- The UI shows player or team score.
- The UI provides feedback after actions.

Backend API

- APIs exist for session creation, joining, topology retrieval, game state, and action submission.
- Backend rules determine outcomes.
- Invalid API requests are handled safely.

Deployment

- The application runs locally using Docker.
- Kubernetes files are provided.
- Health checks are configured.
- Deployment instructions are documented.

Quality and Testing

- Automated tests run with a single command.
- SonarQube analysis runs successfully.
- Quality gate is met or justified.

- Test and quality results are documented.

Stretch Goals

Gameplay Stretch Goals

- Add special packet types.
- Add power-ups or emergency actions.
- Add hidden network faults.
- Add route prediction.
- Add player roles such as Routing Engineer, Capacity Planner, and Incident Responder.
- Add a leaderboard.

O-RAN / Telecom Stretch Goals

- Add simplified O-CU, O-DU, and O-RU nodes.
- Add SMO policy decisions.
- Add Near-RT RIC-inspired traffic steering.
- Add slice-aware routing.
- Add SLA classes for different traffic types.

Technical Stretch Goals

- Add message queues.
- Add distributed tracing.
- Add Prometheus metrics.
- Add Grafana dashboards.
- Add load testing.
- Add replay mode.
- Add pathfinding visualisation.

AI Stretch Goals

- Add an AI route advisor with confidence levels.
- Ask players to approve or reject AI route suggestions.
- Log AI recommendations and final player decisions.
- Add intentionally incomplete AI advice to teach critical review.
- Ensure AI cannot directly change network state.

Expected Deliverables

Each team should deliver source code, browser-playable application, Docker configuration, Kubernetes manifests, automated tests, SonarQube report, architecture document, AI Usage Log, security notes, and final demonstration.

Final Demonstration

The final demo should show creating a packet arena session, multiple players joining, topology display, packet traffic generation, routing action submission, network state and score update, tests, SonarQube results, deployment, and controlled AI usage.

Definition of Done

The project is considered done when:

- The game is playable from a browser.
- Multiple players can participate.
- Packet traffic is simulated.
- Routing actions affect the game state.
- Scores update correctly.
- Docker support exists.
- Kubernetes files exist.
- Automated tests pass.
- SonarQube quality gate is met or justified.
- Security and scalability are documented.
- AI usage is logged and reviewed.